

PC2130 Assignment - Quantum Bouncer: A Python-Based Simulation

Soham Bhar

Course Code: PC2130

E1471301@u.nus.edu

November 12, 2025

Contents

1	Introduction (Part a)	3
2	Time-Independent Solution (Part b)	3
2.1	Scaling and the Airy Equation	4
2.2	Eigenvalues and Eigenfunctions	5
2.3	Python Analysis	6
3	Initial Energy Expectation (Part c)	7
4	Initial State Justification (Part d)	8
4.1	The Normalization Integral	8
4.2	Python Analysis of Normalization	9
4.2.1	Analysis of Python Output	10
4.3	Ascertaining the Condition with $\langle E \rangle$	10
5	Time-Dependent Wavefunction (Part e)	11
5.1	Theoretical Foundation	11
5.2	Calculating Expansion Coefficients	12
5.3	Python Implementation Strategy	12
5.4	Full Python Implementation	13
5.5	Conclusion for Part (e)	16
6	Animation and Visualization (Part f)	16
7	Conclusion	19
A	Graphs generated through Python	19

1 Introduction (Part a)

(a) Schrödinger Equation for the Quantum Bouncer

In this assignment, we will discuss the one-dimensional motion of a quantum particle in a box under the effect of gravity.

Using Φ to represent the wavefunction of the particle, the Schrödinger equation for the quantum particle reads:

$$i\hbar \frac{\partial \Phi}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2 \Phi}{\partial z^2} + V(z)\Phi$$

Here we consider the z -direction and the potential function $V(z)$ is given by

$$V(z) = \begin{cases} mgz, & z > 0 \\ \infty, & z \leq 0 \end{cases}$$

Clearly, for $z < 0$, the only solution to the Schrödinger equation in this context is

$$\Phi = 0$$

Therefore, for $z > 0$, the Schrödinger equation reads:

$$i\hbar \frac{\partial \Phi}{\partial t} = -\frac{\hbar^2}{2m} \frac{\partial^2 \Phi}{\partial z^2} + mgz \Phi \quad (i)$$

subject to the boundary condition:

$$\Phi(0, t) = 0 \quad (z = 0)$$

2 Time-Independent Solution (Part b)

Here, I will solve the TISE by scaling the variables to turn it into the Airy differential equation. I will determine the energy eigenvalues E_n by finding the zeros of the Airy function and derive the normalized energy eigenfunctions $\psi_n(z)$.

For the partial differential equation (PDE) in equation (i), we assume a solution of the form:

$$\Phi(z, t) = \psi(z)\phi(t)$$

This is the classical separation of variables method. Substituting into (i), we get:

$$i\hbar \psi(z) \frac{\partial \phi(t)}{\partial t} = -\frac{\hbar^2}{2m} \frac{\phi(t) \partial^2 \psi(z)}{\partial z^2} + mgz \psi(z) \phi(t)$$

Dividing both sides by $\psi(z)\phi(t)$, we get:

$$\frac{i\hbar}{\phi(t)} \frac{\partial \phi(t)}{\partial t} = -\frac{\hbar^2}{\psi(z)2m} \frac{\partial^2 \psi(z)}{\partial z^2} + mgz$$

This is only possible if both sides are equal to a constant, which we will call E . Therefore, we have two ordinary differential equations (ODEs):

$$\frac{i\hbar}{\phi(t)} \frac{\partial \phi(t)}{\partial t} = E \quad (ii)$$

$$-\frac{\hbar^2}{\psi(z)2m} \frac{\partial^2 \psi(z)}{\partial z^2} + mgz = E \quad (\text{iii})$$

(ii) has the general solution:

$$\phi(t) = Ae^{-iEt/\hbar}$$

(iii) reduces to

$$\frac{-\hbar^2}{2m} \frac{\partial^2 \psi(z)}{\partial z^2} + mgz\psi(z) = E\psi(z) \quad (\text{iv})$$

This is the energy eigenvalue equation. We make the change $E \rightarrow E_n$ and $\psi(z) \rightarrow \psi_n(z)$ since (iv) has infinitely many solutions. Thus, E_n are the eigenvalues and $\psi_n(z)$ are the corresponding eigenfunctions.

2.1 Scaling and the Airy Equation

Now, (iv) has coefficients with dimensions, we want to change it to a dimensionless form since that will allow us to express solutions in the form of infinite series. To simplify the equation, we introduce dimensionless variables. We define

$$l_g = \left(\frac{\hbar^2}{2gm^2} \right)^{1/3}$$

and the dimensionless variables:

$$\zeta = \frac{z}{l_g}$$

$$\epsilon = \frac{E}{mgl_g} = E \left(\frac{2}{\hbar^2 mg^2} \right)^{1/3}$$

Substituting these into the eigenvalue equation:

$$\frac{d^2 \psi_n}{d\zeta^2} + (\epsilon_n - \zeta)\psi_n = 0$$

with boundary condition $\psi_n(0) = 0$.

Solution using Airy Functions

The differential equation:

$$\frac{d^2 \psi_n}{d\zeta^2} - (\zeta - \epsilon_n)\psi_n = 0 \quad (\text{v})$$

is the standard Airy equation with argument $(\zeta - \epsilon_n)$. The general solution is:

$$\psi_n(\zeta) = A_n \text{Ai}(\zeta - \epsilon_n) + B_n \text{Bi}(\zeta - \epsilon_n)$$

where Ai and Bi are the Airy functions of the first and second kind, respectively and A_n and B_n are normalisation constants

Since $\text{Bi}(x) \rightarrow \infty$ as $x \rightarrow \infty$, and we require normalizable solutions, we must have $B_n = 0$. Thus:

$$\psi_n(\zeta) = A_n \text{Ai}(\zeta - \epsilon_n) \quad (\text{vi})$$

Boundary Condition and Quantization

Applying the boundary condition $\psi_n(0) = 0$:

$$A_n \text{Ai}(-\epsilon_n) = 0$$

Since $A_n \neq 0$ (for non-trivial solutions), we require:

$$\text{Ai}(-\epsilon_n) = 0$$

Here we note that the zeroes of the Airy function only occur for negative arguments

Let a_n denote the n -th zero of the Airy function $\text{Ai}(x)$, where $a_1 < a_2 < a_3 < \dots$ and all $a_n > 0$. Then:

$$-\epsilon_n = -a_n \quad \Rightarrow \quad \epsilon_n = a_n$$

2.2 Eigenvalues and Eigenfunctions

Energy Eigenvalues

Converting back to dimensional units:

$$E_n = a_n \cdot mgl_g = a_n \left(\frac{\hbar^2 g^2 m}{2} \right)^{1/3}$$

where a_n is the n -th zero of the Airy function.

For large n , the asymptotic approximation for the zeros is:

$$a_n \approx \left[\frac{3\pi}{2} \left(n - \frac{1}{4} \right) \right]^{2/3} \quad (\text{vii})$$

Normalized Eigenfunctions

The normalized eigenfunctions are:

$$\psi_n(z) = N_n \text{Ai} \left(\frac{z}{l_g} - a_n \right) \quad (\text{viii})$$

where the normalization constant is:

$$N_n = \left[\int_0^\infty \text{Ai}^2 \left(\frac{z}{l_g} - a_n \right) dz \right]^{-1/2}$$

For large n , an approximate expression is:

$$N_n \approx \left(\frac{\pi}{\sqrt{a_n}} \right)^{1/2} \approx \left[\frac{2\pi^2}{3(n - 1/4)} \right]^{1/6} \quad (\text{ix})$$

Orthogonality

The eigenfunctions are mutually orthogonal because they correspond to different eigenvalues of the Hermitian operator $\hat{H}$. This follows from the general theorem that eigenfunctions of a Hermitian operator corresponding to different eigenvalues are orthogonal:

$$\int_0^\infty \psi_n^*(z) \psi_m(z) dz = \delta_{nm}$$

where δ_{nm} is the Kronecker delta.

Physical Interpretation

Each eigenfunction represents a stationary state of the quantum particle bouncing in the gravitational field. The energy levels are discrete and non-equally spaced. The ground state ($n = 1$) has energy $E_1 = a_1(mg\hbar^2/2)^{1/3}$, and higher excited states correspond to the particle being "dropped" from progressively greater effective heights.

2.3 Python Analysis

The following python code is used to plot n number of eigenvalues and corresponding eigenfunctions. The sympy library has built in functions to calculate the Airy zeroes so we do not need to use the approximate expressions given.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import airy, ai_zeros

# Define Airy parameters
n_states = 5 # number of eigenstates to plot
z = np.linspace(0, 20, 500)

# Get first few zeros of the Airy Ai function
# ai_zeros returns the negative roots of Ai(x) [2, 3, 4, 5, 6, 7]
zeros, _, _ = ai_zeros(n_states)
# The scaled energies  $E_n'$  are positive
E_n_prime = -zeros

# Compute eigenfunctions  $\psi_n(z')$ 
psi = []
for n in range(n_states):
    # airy(x) returns the Ai(x) function [8]
    psi_n = airy(z - E_n_prime[n])[0] # Ai(z -  $E_n'$ )
    # Normalize the eigenfunction using trapezoidal integration
    psi.append(psi_n / np.sqrt(np.trapz(psi_n**2, z)))

# Plot results
plt.figure(figsize=(8, 6))

# Add a horizontal line at y=0 to highlight the zeros
plt.axhline(0, color='black', linestyle='--', lw=1)

for n in range(n_states):
    # Plot without the y-axis shift
    plt.plot(z, psi[n], label=f"$n={n+1}$, $E'_n={E_n_prime[n]:.3f}$")

plt.title("Solutions to the Airy Equation  $\psi'' - (z' - E'_n)\psi = 0$ ")
plt.xlabel("$z'$ (dimensionless height)")
plt.ylabel("$\psi_n(z')$") # Updated label
plt.legend()
plt.grid(True)
plt.show()
```

Note that in the code z_n has been used instead of a_n and E'_n instead of ϵ_n . This is due to ease of declaration of variables in python and also because latex does not (by default) support math characters in verbatim environment.

3 Initial Energy Expectation (Part c)

We have to calculate the expectation value of energy $\langle E \rangle$ for the initial unbound Gaussian wave packet.

Given the initial wave function:

$$\psi_0(z) = \frac{1}{\sqrt{\sqrt{2\pi}\sigma_0}} \exp \left[-\frac{(z - z_0)^2}{4\sigma_0^2} \right]$$

Which is a standard Gaussian wave packet. We also note that ψ_0 is real, thus $\psi_0(z) = \psi_0^*(z)$

The expectation value of energy is:

$$\langle E \rangle = \langle \psi_0 | \hat{H} | \psi_0 \rangle = \int_{-\infty}^{\infty} \psi_0^*(z) \hat{H} \psi_0(z) dz$$

where $\hat{H}$ represents the Hamiltonian operator:

$$\hat{H} = -\frac{\hbar^2}{2m} \frac{d^2}{dz^2} + mgz$$

We thus have,

$$\langle E \rangle = \int_{-\infty}^{\infty} \psi_0^*(z) \left(-\frac{\hbar^2}{2m} \frac{d^2}{dz^2} + mgz \right) \psi_0(z) dz$$

We can split this into kinetic and potential energy contributions:

$$\langle E \rangle = \langle T \rangle + \langle V \rangle$$

Kinetic Energy Term:

$$\langle T \rangle = \int_{-\infty}^{\infty} \psi_0^*(z) \left(-\frac{\hbar^2}{2m} \frac{d^2}{dz^2} \right) \psi_0(z) dz \quad (1)$$

First, we calculate the second derivative of $\psi_0(z)$:

$$\frac{d\psi_0}{dz} = \psi_0(z) \cdot \left(-\frac{z - z_0}{2\sigma_0^2} \right) \quad (2)$$

$$\frac{d^2\psi_0}{dz^2} = \psi_0(z) \left[\left(-\frac{z - z_0}{2\sigma_0^2} \right)^2 - \frac{1}{2\sigma_0^2} \right] = \psi_0(z) \left[\frac{(z - z_0)^2}{4\sigma_0^4} - \frac{1}{2\sigma_0^2} \right] \quad (3)$$

Therefore:

$$\langle T \rangle = -\frac{\hbar^2}{2m} \int_{-\infty}^{\infty} |\psi_0(z)|^2 \left[\frac{(z - z_0)^2}{4\sigma_0^4} - \frac{1}{2\sigma_0^2} \right] dz \quad (4)$$

$$= -\frac{\hbar^2}{2m} \left[\frac{1}{4\sigma_0^4} \langle (z - z_0)^2 \rangle - \frac{1}{2\sigma_0^2} \right] \quad (5)$$

Here we have imposed the normalisation condition for $\psi_0(z)$ for the second term.

$$\int_{-\infty}^{\infty} |\psi_0(z)|^2 dz = 1 \quad (6)$$

For the Gaussian wave packet, $\langle (z - z_0)^2 \rangle = 2\sigma_0^2$, so:

$$\langle T \rangle = -\frac{\hbar^2}{2m} \left[\frac{2\sigma_0^2}{4\sigma_0^4} - \frac{1}{2\sigma_0^2} \right] = -\frac{\hbar^2}{2m} \left[\frac{1}{2\sigma_0^2} - \frac{1}{2\sigma_0^2} \right] + \frac{\hbar^2}{8m\sigma_0^2} \quad (7)$$

$$\langle T \rangle = \frac{\hbar^2}{8m\sigma_0^2} \quad (8)$$

Potential Energy Term:

$$\langle V \rangle = \int_{-\infty}^{\infty} |\psi_0(z)|^2 \cdot mgz \, dz = mg\langle z \rangle \quad (9)$$

For the Gaussian centered at z_0 :

$$\langle z \rangle = z_0 \quad (10)$$

Therefore:

$$\langle V \rangle = mgz_0 \quad (11)$$

Total Energy:

$$\boxed{\langle E \rangle = \frac{\hbar^2}{8m\sigma_0^2} + mgz_0} \quad (12)$$

This result represents the sum of the kinetic energy due to the quantum mechanical uncertainty in momentum (proportional to $1/\sigma_0^2$) and the potential energy at the center of the wave packet.

4 Initial State Justification (Part d)

We need to:

1. Calculate the normalization integral $I = \int_0^{\infty} |\psi_0(z)|^2 dz$ and use it to determine a condition for σ_0 and z_0 such that $I \approx 1$.
2. Ascertain this condition by analyzing the expectation value $\langle E \rangle$ over the new, bounded domain $0 < z < \infty$.

The guiding principle is to find the condition under which the wave packet is almost entirely ($> 99.9\%$) in the physical region $z > 0$, making it a valid, normalized initial state for the "bouncer" problem.

4.1 The Normalization Integral

We must first calculate the integral $I = \int_0^{\infty} |\psi_0(z)|^2 dz$. As established in part (c), the integrand $|\psi_0(z)|^2$ is a standard Gaussian distribution $N(\mu, \sigma)$ with mean $\mu = z_0$ and standard deviation $\sigma = \sigma_0$.

The integral I is the cumulative probability of this Gaussian from $z = 0$ to $z = \infty$. As hinted in the assignment, this is analytically solvable using the error function, $\text{erf}(x)$. The general solution for this integral is:

$$I = \int_0^{\infty} \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp \left[-\frac{(z - z_0)^2}{2\sigma_0^2} \right] dz = \frac{1}{2} \left[1 + \text{erf} \left(\frac{z - z_0}{\sigma_0\sqrt{2}} \right) \right] \Bigg|_0^{\infty}$$

Evaluating this at the limits:

- As $z \rightarrow \infty$, the term $\text{erf}(\infty) \rightarrow 1$.
- At $z = 0$, the term is $\text{erf}\left(\frac{-z_0}{\sigma_0\sqrt{2}}\right) = -\text{erf}\left(\frac{z_0}{\sigma_0\sqrt{2}}\right)$.

Plugging these in gives:

$$I = \frac{1}{2} \left[(1 + 1) - \left(1 - \text{erf}\left(\frac{z_0}{\sigma_0\sqrt{2}}\right) \right) \right] = \frac{1}{2} \left[1 + \text{erf}\left(\frac{z_0}{\sigma_0\sqrt{2}}\right) \right]$$

For $\psi_0(z)$ to be an appropriate description, we need the total probability in the physical domain I to be approximately 1. This requires:

$$\text{erf}\left(\frac{z_0}{\sigma_0\sqrt{2}}\right) \approx 1$$

The error function $\text{erf}(x)$ approaches 1 very quickly as x increases. This gives us our condition:

$$\frac{z_0}{\sigma_0\sqrt{2}} \gg 1 \quad \implies \quad \mathbf{z_0} \gg \sigma_0$$

It is important to note that again, Sympy has built in functions to generate solution of the integral. Thus we need not know the general solution to create the plot using python.

4.2 Python Analysis of Normalization

To visualize and quantify this condition, the following Python script was used. It plots the value of the integral I as a function of the ratio $r = z_0/\sigma_0$, using `scipy.special.erf` (the Python equivalent of Mathematica's `Erf`).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erf

def calculate_normalization_I(z0, sigma0):
    """
    Calculates the integral of |psi_0|^2 from 0 to infinity.

    This is the probability of finding the particle in the
    physical domain z > 0.
    """
    argument = z0 / (sigma0 * np.sqrt(2.0))
    integral_value = 0.5 * (1.0 + erf(argument))
    return integral_value

# --- Create the plot to determine the condition ---

# We only care about the ratio of z0 to sigma0.
# Let's create an array of ratios r = z0 / sigma0
r_values = np.linspace(0, 5, 200)

# Calculate the integral I for each ratio
```

```

# (We can just set sigma0 = 1 and vary z0)
I_values = calculate_normalization_I(r_values, 1.0)

# --- Plotting the results ---
plt.figure(figsize=(10, 6))
plt.plot(r_values, I_values, lw=2)
plt.axhline(y=1.0, color='gray', linestyle='--',
            label='$I = 1$ (Full Normalization)')
plt.axvline(x=3.0, color='r', linestyle=':',
            label='$z_0 / \sigma_0 = 3$')
I_at_3 = calculate_normalization_I(3.0, 1.0)
plt.axhline(y=I_at_3, color='r', linestyle=':')
plt.text(0.5, I_at_3 - 0.01,
         f'$I = \{I\_at\_3:.5f\}$ (99.87% normalized)', color='r')

plt.title('Normalization Integral $I = \int_0^{\infty} |\psi_0(z)|^2 dz$',
          fontsize=16)
plt.xlabel('Ratio $z_0 / \sigma_0$', fontsize=12)
plt.ylabel('Integral Value (Total Probability)', fontsize=12)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.ylim(0.4, 1.02)
plt.xlim(0, 5)

print(f"I(r=1): {calculate_normalization_I(1.0, 1.0):.4f}")
print(f"I(r=2): {calculate_normalization_I(2.0, 1.0):.4f}")
print(f"I(r=3): {calculate_normalization_I(3.0, 1.0):.4f}")

plt.show()

```

4.2.1 Analysis of Python Output

The script's printed output provides a clear quantitative measure:

```

I(r=1): 0.8413
I(r=2): 0.9772
I(r=3): 0.9987

```

The plot and this output show that when z_0 is only one standard deviation away from the floor ($r = 1$), over 15% of the wave packet is "missing". By the time the ratio is $r = 3$, 99.87% of the wave packet is in the correct region. **The condition is $z_0/\sigma_0 \geq 3$ (or more generally, $z_0 \gg \sigma_0$).**

4.3 Ascertaining the Condition with $\langle E \rangle$

We must ascertain the condition by calculating $\langle E \rangle$ again, but over $0 < z < \infty$."

The logic is as follows:

1. From Part (c), the expectation value of energy for the *unbounded* problem is $\langle E \rangle_{\text{unbounded}} = \frac{\hbar^2}{8m\sigma_0^2} + mgz_0$.

2. The energy for the *bounded* problem is $\langle E \rangle_{\text{bounded}} = \int_0^\infty \psi_0^*(z) \hat{H} \psi_0(z) dz$.
3. The question is, under what condition is $\langle E \rangle_{\text{bounded}} \approx \langle E \rangle_{\text{unbounded}}$?
4. The difference between the two is the integral we cut off by introducing the floor:

$$\langle E \rangle_{\text{unbounded}} - \langle E \rangle_{\text{bounded}} = \int_{-\infty}^0 \psi_0^*(z) \hat{H} \psi_0(z) dz$$

5. The condition $z_0 \gg \sigma_0$, which we found from the normalization integral, means that the Gaussian function $\psi_0(z)$ and its derivatives are already practically zero for all $z \leq 0$.
6. Since the Hamiltonian integrand, $\psi_0^*(z) \hat{H} \psi_0(z)$, is composed of $\psi_0(z)$ and its derivatives, this entire expression is also negligible for all $z \leq 0$.

Conclusion: The condition $z_0 \gg \sigma_0$ is ascertained (confirmed) because it is this very condition that ensures the energy expectation value is unchanged. If the condition holds, the particle's initial energy $\langle E \rangle$ is not affected by the introduction of the floor at $z = 0$, because the particle is "unaware" of the floor's existence at $t = 0$. This confirms $\psi_0(z)$ is a valid initial state.

5 Time-Dependent Wavefunction (Part e)

The time-dependent wavefunction $\Psi(z, t)$ is constructed as a superposition of the eigenstates found in Part (b). We recall our initial ansatz

$$\Phi(z, t) = \psi(z) \phi(t)$$

We have,

$$\Phi(z, t) = \sum_{n=1}^{\infty} C_n e^{-iE_n t/\hbar} \psi_n(z)$$

The coefficients C_n will be calculated by projecting the initial state onto the eigenbasis.

This function is constructed by building upon the solutions from all previous parts.

5.1 Theoretical Foundation

From the superposition principle, the solution to the time-dependent Schrödinger equation can be expressed as an expansion in the basis of its energy eigenstates, $\psi_n(z)$. This is given by Eq. (4) in the Gea-Banacloche paper:

$$\Phi(z, t) = \sum_{n=1}^{\infty} C_n e^{-iE_n t/\hbar} \psi_n(z)$$

To implement this numerically, we use the dimensionless scaled variables for position (z'), energy ($E'_n = z_n$), and time (t') as defined in part (b) and the reference paper. The scaled equation is:

$$\Phi(z', t') = \sum_{n=1}^{\infty} C_n e^{-iz_n t'} \psi_n(z')$$

where:

- $\psi_n(z') = \mathcal{N}_n \text{Ai}(z' - z_n)$ are the normalized energy eigenfunctions found in part (b).
- z_n are the scaled energy eigenvalues (the zeros of the Airy function) from part (b).
- C_n are the time-independent expansion coefficients that "weigh" how much of each eigenstate is present in the initial state $\psi(z', 0)$.

Here we also rescale the time since the quantity in the exponent must be dimensionless otherwise it has no physical meaning. Considering our eigenfunctions are dimensionless, we must scale our time

$$t \rightarrow t'$$

accordingly to make the whole expression

$$(-iz_n t')$$

dimensionless

5.2 Calculating Expansion Coefficients

The coefficients C_n are found by projecting the initial state $\Psi(z', 0)$ onto the eigenstate basis $\psi_n(z')$. This is a standard Fourier-style integral

$$C_n = \int_0^\infty \psi_n^*(z') \psi(z', 0) dz'$$

We recall that $\psi_0(z, 0) = \frac{1}{\sqrt{\sqrt{2\pi}\sigma_0}} \exp\left[-\frac{(z-z_0)^2}{4\sigma_0^2}\right]$

Using the functions from our previous work:

- $\psi_n^*(z') = \psi_n(z') = \mathcal{N}_n \text{Ai}(z' - z_n)$ (the eigenfunctions are real).
- ψ_0 is the initial gaussian wave packet. We use the form given above, which is normalised under the condition $z_0 \gg \sigma$ which we found in part d

Substituting these gives the final integral we must compute for each coefficient C_n :

$$C_n = \mathcal{N}_n \frac{1}{\sqrt{\sqrt{2\pi}\sigma_0}} \int_0^\infty \text{Ai}(z - z_n) \exp\left[-\frac{(z - z_0)^2}{4\sigma_0^2}\right] dz$$

This integral must be solved numerically for each value of n .

5.3 Python Implementation Strategy

To solve this in Python, we replace the Mathematica 'Sum' and 'Integrate' functions with their SciPy and NumPy equivalents. An implementation that re-computes these integrals at every time step would be far too slow.

The efficient approach is to pre-compute all time-independent quantities and store them in NumPy arrays. The Sum function is then replaced by a fast, vectorized NumPy dot product.

The workflow is as follows:

1. **Choose Parameters:** Based on part (d) and the paper on the quantum bouncer, we select $z_0 = 20$, $\sigma = 3$, and truncate the infinite sum to a finite $N = 100$ states.

2. Compute Eigenstate Arrays (from part b):

- `zn_array`: A NumPy array of N eigenvalues, calculated using `scipy.special.ai_zeros[5, 6]`
- `Nn_array`: A NumPy array of N normalization constants, calculated by looping N times and calling `scipy.integrate.quad` on the integral for $\mathcal{N}_n$.

3. Compute Coefficient Array (part e):

- `Cn_array`: A NumPy array of N expansion coefficients. This is the main task of part (e), calculated by looping N times and calling `scipy.integrate.quad` on the integral for C_n given above.

4. Define Final $\Psi(z, t)$ Function:

- Create a function `get_psi_zt(z_grid, t)` that takes a grid of positions and a single time t .
- Inside this function, NumPy broadcasting is used to efficiently calculate all terms, and `numpy.dot` is used to perform the sum over n .

5.4 Full Python Implementation

The following Python code implements this entire workflow. It pre-computes all necessary arrays and defines the final `get_psi_zt` function, which is the deliverable for this part and will be used for the animation in part (f).

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import airy, ai_zeros
from scipy.integrate import quad
from matplotlib.animation import FuncAnimation
import time

# === PART (e): PRE-COMPUTATION & FUNCTION DEFINITION ===

# --- Step 1: Choose Simulation Parameters ---
# Based on our analysis in part (d) and the paper ,
# we need  $z_0 \gg \sigma_0$ . We'll use the paper's values.
z0_scaled = 20.0
sigma_scaled = 3.0

# Number of eigenstates (N) to include in the sum.
N_states = 100

print(f"Initializing simulation with N={N_states} states.")
print(f"Initial height  $z_0 = \{z0\_scaled\}$ , width  $\sigma = \{sigma\_scaled\}$ \n")
```

```

# --- Step 2: Compute Eigenvalues (zn) and Normalizations (Nn) ---
# This is the solution from part (b), now run for N_states.

print("Step 2: Computing eigenvalues and normalization constants...")
start_time = time.time()

# 'ai_zeros' returns the *negative* zeros of Ai(x). [4, 5, 6, 7]
# The paper's z_n are the *positive* scaled energies, E_n' = z_n,
# where Ai(-E_n') = Ai(-z_n) = 0.
# Therefore, we must flip the sign: z_n = -a_n
a_n, _, _, _ = ai_zeros(N_states)
zn_array = -a_n

# We must compute the normalization constant Nn for each state:
# N_n = 1 / sqrt( integral_0^inf [Ai(z' - z_n)]^2 dz' )
Nn_array = np.zeros(N_states)

def normalization_integrand(z_prime, zn):
    """Integrand for the normalization constant: Ai(z'-zn)^2"""
    # 'airy' returns a tuple (Ai, Ai', Bi, Bi') [8]
    # We only want Ai, which is at index 0.
    ai_val = airy(z_prime - zn)[0]
    return ai_val**2

for n in range(N_states):
    # 'quad' is the Python equivalent of Mathematica's 'Integrate'
    # for numerical integration. [9, 10]
    integral_val, error = quad(normalization_integrand, 0, np.inf, args=(zn_array[n]))
    Nn_array[n] = 1.0 / np.sqrt(integral_val)

print(f"Step 2 complete in {time.time() - start_time:.2f} seconds.\n")

# --- Step 3: Compute Expansion Coefficients (Cn) ---
# This is the primary task of part (e).
# C_n = integral_0^inf [psi_n(z') * Psi(z', 0)] dz'
# C_n = integral_0^inf [N_n * Ai(z' - z_n)] * [Psi_gaussian(z')] dz'

print("Step 3: Computing expansion coefficients C_n...")
start_time = time.time()

# Define the initial Gaussian wave packet, Psi(z', 0)
# From paper Eq. (15)
gaussian_norm = (2.0 / (np.pi * sigma_scaled**2))**(0.25)

def psi_initial_gaussian(z_prime):
    """ The initial Gaussian wave packet at t=0 """
    exponent = -(z_prime - z0_scaled)**2 / sigma_scaled**2

```

```

    return gaussian_norm * np.exp(exponent)

# Define the integrand for the C_n coefficients
def cn_integrand(z_prime, n):
    """ Integrand for C_n = psi_n(z') * Psi(z', 0) """
    zn = zn_array[n]
    Nn = Nn_array[n]

    # psi_n(z') = N_n * Ai(z' - z_n)
    # We need the Ai value at index 0 [8]
    psi_n_z = Nn * airy(z_prime - zn)[0]

    # Psi(z', 0)
    psi_0_z = psi_initial_gaussian(z_prime)

    return psi_n_z * psi_0_z

# Loop N_states times to calculate all C_n
Cn_array = np.zeros(N_states)
for n in range(N_states):
    integral_val, error = quad(cn_integrand, 0, np.inf, args=(n))
    Cn_array[n] = integral_val

print(f"Step 3 complete in {time.time() - start_time:.2f} seconds.\n")

# --- Step 4: Define the Time-Dependent Wave Function Psi(z, t) ---
# This function is the final deliverable for part (e).
# It replaces the Mathematica 'Sum' function with
# a fast, vectorized NumPy implementation. [11, 3, 12, 13]

print("Step 4: Defining the vectorized function Psi(z, t).")

def get_psi_zt(z_prime_grid, t_prime):
    """
    Calculates the time-dependent wave function Psi(z', t')
    at all positions in z_prime_grid for a given time t_prime.
    """

    # Use NumPy broadcasting to create an (N, M) matrix of (z' - z_n)
    z_minus_zn = z_prime_grid[np.newaxis, :] - zn_array[:, np.newaxis]

    # 1. Get all eigenfunction values: psi_n(z') = N_n * Ai(z' - z_n)
    # This will be an (N, M) matrix.
    # We need the Ai value at index 0 [8]
    psi_n_z_matrix = Nn_array[:, np.newaxis] * airy(z_minus_zn)[0]

    # 2. Get all time-evolved coefficients: C_n(t') = C_n(0) * exp(-i*z_n*t')

```

```

# This will be a (N,) array.
time_evolution = np.exp(-1j * zn_array * t_prime)
Cn_t_array = Cn_array * time_evolution

# 3. Perform the sum: Psi(z', t') = SUM_n [ C_n(t') * psi_n(z') ]
# This is a dot product between the (N,) array and the (N, M) matrix.
Psi_at_t = np.dot(Cn_t_array, psi_n_z_matrix)

return Psi_at_t

print("Part (e) solution is complete.")
print("The function 'get_psi_zt(z_grid, t)' is defined and ready for part (f).")

```

5.5 Conclusion for Part (e)

The solution to part (e) is the Python function `get_psi_zt`, along with the pre-computed arrays `zn_array`, `Nn_array`, and `Cn_array` that it depends on. This function can now be used to compute the wave function $\Psi(z', t')$ at any position and time, and will serve as the core engine for the animation in part (f).

6 Animation and Visualization (Part f)

This section contains the visualizations of the quantum bouncer's dynamics, including plots of $\Psi(z, t)$ and the probability density $|\Psi(z, t)|^2$ over time, created using Python. The following code was added to the code from part(e) to create the animations.

```

print("\n--- Part (f): Creating Animation ---")

# 1. Setup the simulation grid and time parameters
z_grid = np.linspace(0, 45, 500) # Scaled position grid (M=500 points)
N_frames = 400                    # Total number of frames for the movie
dt_scaled = 0.5                   # Scaled time step per frame
t_total_scaled = N_frames * dt_scaled
print(f"Animation will run for {N_frames} frames.")
print(f"Total scaled time: {t_total_scaled:.1f}")

# 2. Setup the figure and axes for plotting
fig, ax = plt.subplots(figsize=(10, 6))

# Set fixed plot limits for a stable animation
ax.set_xlim(0, 45)
ax.set_ylim(-0.6, 0.6) # Set y-limits to fit both real part and probability

# Add labels and title
ax.set_title(f'Quantum Bouncer Dynamics ($z_0={z0_scaled}$, \sigma={sigma_scaled}$)',
ax.set_xlabel("Scaled Position $z$'", fontsize=12)
ax.set_ylabel("Amplitude / Probability Density", fontsize=12)

```



```

# 3. Initialize the plot artists (the lines that will be animated)

# Get the t=0 state using our function from part (e)
psi_t0 = get_psi_zt(z_grid, 0.0)
prob_t0 = np.abs(psi_t0)**2
real_t0 = np.real(psi_t0)

# Create the line objects
line_prob, = ax.plot(z_grid, prob_t0, 'r-',
                    label='$|\Psi(z',t')|^2$ (Probability)')
line_real, = ax.plot(z_grid, real_t0, 'b-',
                    label='Re[$\Psi(z',t')$] (Wave Function)',
                    alpha=0.7)

# Add a text display for the current time
time_text = ax.text(0.7, 0.9, '', transform=ax.transAxes, fontsize=12)
ax.legend(loc='lower right')
ax.grid(True, linestyle='--', alpha=0.6)

# 4. Define the update function (called for each frame)
# This is the core of the animation
def update(frame):
    """
    Updates the plot data for each new frame.
    """
    t_scaled = frame * dt_scaled

    # Get the new wavefunction from our part (e) function
    Psi_t = get_psi_zt(z_grid, t_scaled)

    # Calculate the new data for the plots
    Prob_t = np.abs(Psi_t)**2
    Real_t = np.real(Psi_t)

    # Update the data of the line artists
    line_prob.set_ydata(Prob_t)
    line_real.set_ydata(Real_t)
    time_text.set_text(f'Scaled time $t$' = {t_scaled:.2f}$')

    # Return all artists that were updated
    return line_prob, line_real, time_text

# 5. Create and save the animation
# 'FuncAnimation' is the Python equivalent of Mathematica's 'Animate'
print(f"Generating animation with {N_frames} frames...")

```

```

print("This may take a few minutes...")
start_anim_time = time.time()

anim = FuncAnimation(fig,                # The figure to animate
                    update,              # The function to call for each frame
                    frames=N_frames,     # Total number of frames
                    interval=30,         # Milliseconds per frame (e.g., 30ms)
                    blit=True)           # Use blitting for performance

# Save the animation as an mp4 file
# Note: This requires the 'ffmpeg' video codec to be installed on your system.
try:
    anim.save('quantum_bouncer_animation.mp4', writer='ffmpeg', dpi=150)
    end_anim_time = time.time()
    print(f"\nAnimation successfully saved to 'quantum_bouncer_animation.mp4'")
    print(f"Animation generation took {end_anim_time - start_anim_time:.2f} seconds.")
except Exception as e:
    print(f"\nError saving animation. Do you have 'ffmpeg' installed?")
    print(f"Error: {e}")

# If you are in an interactive environment (like a script),
# you might want to show the plot window:
plt.show()

```

Link to a sped up version of the animation is present at the end of the appendix.

7 Conclusion

In this assignment, we have studied the quantum bouncer — a quantum particle under the influence of gravity confined by an infinite potential wall. By solving the Schrödinger equation using Airy functions, we obtained discrete energy eigenvalues determined by the zeros of the Airy function, and verified the corresponding normalized eigenstates. The system demonstrates how quantization naturally arises even in a simple gravitational potential. Through numerical computation and visualization, we also explored the time evolution of the wavefunction and its probability density. Overall, this exercise illustrates the close connection between mathematical structure and physical behavior in quantum systems, and highlights the usefulness of dimensionless scaling and computational tools in modern quantum analysis.

A Graphs generated through Python

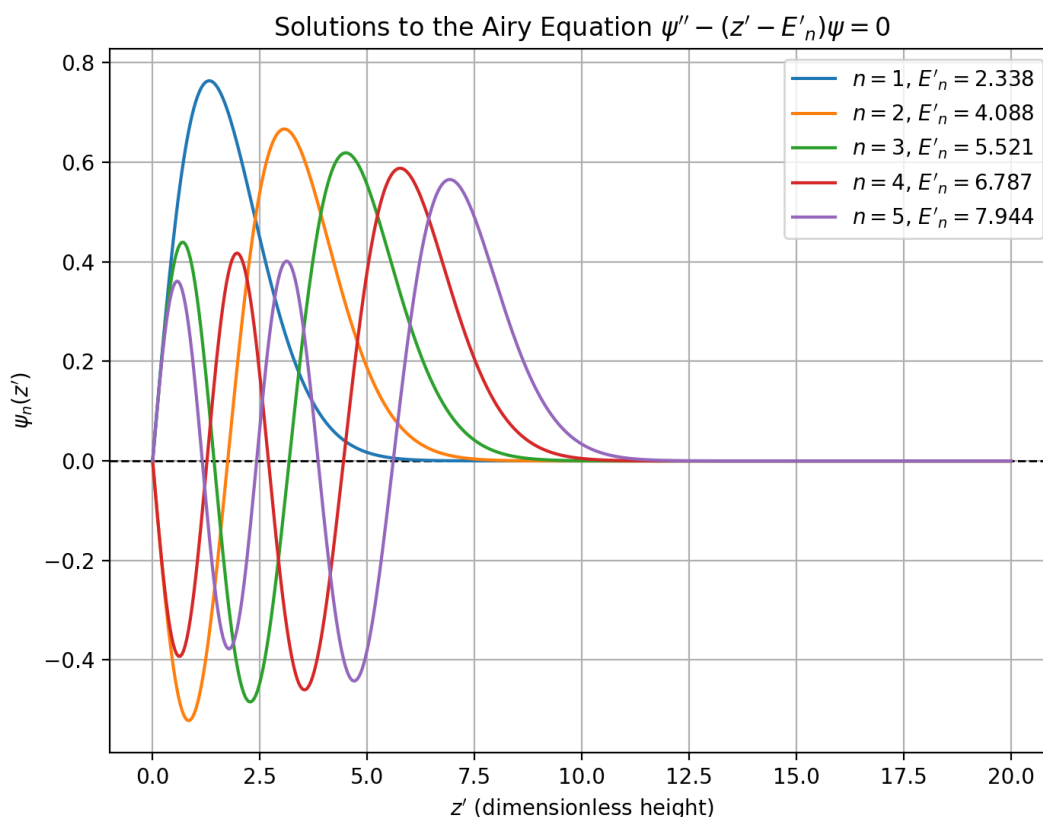


Figure 1: From part 2, First 5 eigenfunctions graphed

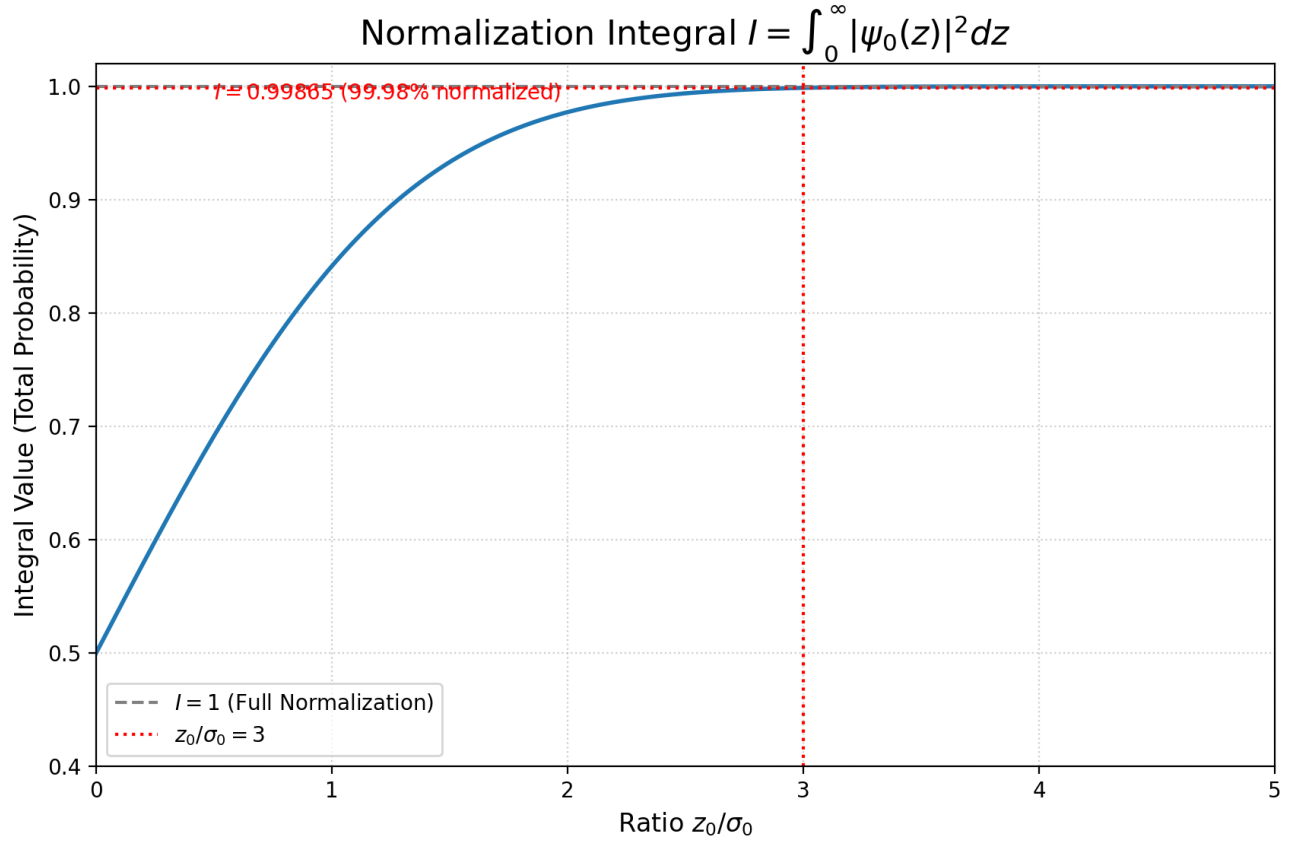


Figure 2: From part 4, Graph of the integrand I against the ratio $r = \frac{r_0}{\sigma}$. As we can see from the graph, as r becomes greater than 3, the value of the integrand rapidly approaches 1. The integrand indicates total probability. This means that once the ratio exceeds 3 almost the entire wave packet is in the region $z \leq 0$. Thus, this graph helps us determine an appropriate value for z_0 and σ so that $\psi_0(z, 0)$ describes a valid initial condition.

[Here is a link to the animation for part f.](#) Note that it is saved as a GIF to optimise storage. Hence, the animation is significantly sped up.